Abstraction in C++

1. What is Abstraction?

Abstraction means hiding unnecessary implementation details and showing only the essential features to the user.

Real-Life Example: Car 🚗

When you drive a car:

- You use the steering wheel to turn.
- You press the accelerator to increase speed.
- You do not need to understand how the engine works internally.

This is **abstraction** — you know *what to do* without needing to know *how it works internally*.

2. How to Achieve Abstraction in C++?

We can achieve abstraction using:

1. **Classes** — expose necessary public functions while hiding internal details.
2. **Access specifiers** — use `private` to hide data and `public` to provide access.
3. **Abstract classes** — use pure virtual functions to define a common interface.

3. Simple Example

```
#include <iostream>
using namespace std;

class CoffeeMachine {
private:
    void heatWater() {
        cout << "Heating water..." << endl;
    }

public:
    void makeCoffee() {
        heatWater();
        cout << "Coffee is ready!" << endl;
    }
};

int main() {
    CoffeeMachine c;
    c.makeCoffee();
```

```
    return 0;
}
```

Output:

```
Heating water...
Coffee is ready!
```

Explanation:

- `heatWater()` is private, so it cannot be called directly from `main()`.
- `makeCoffee()` is public, so the user can call it.
- The user only needs to call `makeCoffee()` to prepare coffee.

4. Abstract Class in C++

An **abstract class** is a class that cannot be used to create objects directly.

It contains at least one **pure virtual function**.

Syntax

```
class Animal {
public:
    virtual void sound() = 0;
};
```

Here:

- `virtual` allows a function to be overridden in derived classes.
- `= 0` makes the function a pure virtual function.
- `Animal` becomes an abstract class.

5. Abstract Class Example

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void sound() = 0;
};

class Dog : public Animal {
public:
    void sound() override {
```

```
        cout << "Dog barks!" << endl;
    }
};

int main() {
    // Animal a;  // Error: abstract class

    Dog d;
    d.sound();

    return 0;
}
```

Output:

```
Dog barks!
```

Remember:

- We cannot create an object of `Animal`.
- We can create an object of `Dog`.
- A derived class must implement the inherited pure virtual function to become a concrete class.

6. Abstraction vs Encapsulation

Abstraction	Encapsulation
Hides implementation details.	Bundles data and methods together and controls access.
Focuses on what an object does.	Focuses on how data and methods are organized and protected.
Example: `makeCoffee()` hides the preparation steps.	Example: keeping a bank account balance `private`.

7. Quick Revision

- Abstraction = showing essential features and hiding implementation details.
- `private` hides members from direct outside access.
- An abstract class has at least one pure virtual function.
- A pure virtual function uses `= 0`.
- Abstract classes cannot be instantiated directly.
- Derived classes can implement pure virtual functions.

Practice Questions 🧠

Level 1 — Basic

Q1. What is abstraction? Explain with one real-life example.

Q2. Fill in the blank:

A pure virtual function is declared using _____.

Q3. True or False: We can create an object of an abstract class.

Q4. Identify the pure virtual function:

```
class Shape {
public:
    virtual void draw() = 0;

    void display() {
        cout << "Shape";
    }
};
```

Level 2 — Predict the Output

Q5. What is the output?

```
#include <iostream>
using namespace std;

class Remote {
private:
    void connect() {
        cout << "Connected!" << endl;
    }

public:
    void turnOn() {
        connect();
        cout << "TV is ON!" << endl;
    }
};

int main() {
    Remote r;
    r.turnOn();
}
```

Q6. Will this code compile? Why or why not?

```
class Vehicle {
public:
```

```
    virtual void start() = 0;
};

int main() {
    Vehicle v;
}
```

Level 3 — Coding Challenges

Q7. Build a Shape System

Create an abstract class `Shape` with a pure virtual function `area()`.

Create two derived classes:

- `Rectangle` — calculate area using `length * width`.
- `Square` — calculate area using `side * side`.

Print the area of both shapes.

Sample output:

```
Rectangle Area: 20
Square Area: 25
```

Q8. Build a Smart Device System

Create an abstract class `SmartDevice` with these pure virtual functions:

```
virtual void turnOn() = 0;
virtual void turnOff() = 0;
```

Create a `SmartFan` class that implements both functions.

Expected output:

```
Smart Fan is ON!
Smart Fan is OFF!
```

Challenge: Try Q7 and Q8 without looking at the earlier examples.